

```

"""
logging_utils.py
=====

Logging del proceso. Cumple dos funciones:

1. Mostrar el avance en consola y persistirlo en un archivo `.log`.
2. Capturar TODOS los mensajes en memoria para volcarlos despues a la hoja
   `08_Bitacora_Log` del Excel. Asi la bitacora es autocontenida: quien reciba
   el Excel puede reconstruir la corrida sin pedir el archivo de log.
"""

from __future__ import annotations

import logging
import sys
from datetime import datetime
from pathlib import Path

#: Formato unico para consola y archivo.
_FORMATO = "%(asctime)s | %(levelname)-7s | %(name)-28s | %(message)s"
_FECHA = "%Y-%m-%d %H:%M:%S"

class ManejadorMemoria(logging.Handler):
    """Handler que acumula los registros en una lista de diccionarios.

    Se usa como fuente de la hoja de bitacora del Excel.
    """

    def __init__(self) -> None:
        super().__init__()
        self.registros: list[dict[str, str]] = []

    def emit(self, record: logging.LogRecord) -> None: # noqa: D102
        try:
            self.registros.append(
                {
                    "timestamp": datetime.fromtimestamp(record.created).strftime(_FECHA),
                    "nivel": record.levelname,
                    "modulo": record.name,
                    "mensaje": record.getMessage(),
                }
            )
        except Exception: # noqa: BLE001 - el logging nunca debe tumbar el proceso
            self.handleError(record)

def configurar_logging(
    ruta_log: str | Path | None = None,
    nivel: int = logging.INFO,
) -> ManejadorMemoria:
    """Configura el logger raiz `featsel` y devuelve el handler en memoria.

    Parameters
    -----
    ruta_log
        Archivo donde persistir el log. Si es ``None`` solo se usa consola.
    nivel
        Nivel minimo (`logging.INFO` por defecto; `DEBUG` para auditoria fina).
    """
    logger = logging.getLogger("featsel")
    logger.setLevel(nivel)
    logger.handlers.clear() # evita duplicar salidas al reejecutar
    logger.propagate = False

    formatter = logging.Formatter(_FORMATO, datefmt=_FECHA)

    consola = logging.StreamHandler(stream=sys.stdout)
    consola.setFormatter(formatter)
    consola.setLevel(nivel)

```

```
logger.addHandler(console)
```

```
if ruta_log is not None:
```

```
    ruta_log = Path(ruta_log)
```

```
    ruta_log.parent.mkdir(parents=True, exist_ok=True)
```

```
    archivo = logging.FileHandler(ruta_log, mode="w", encoding="utf-8")
```

```
    archivo.setFormatter(formatter)
```

```
    archivo.setLevel(logging.DEBUG)
```

```
    logger.addHandler(archivo)
```

```
memoria = ManejadorMemoria()
```

```
memoria.setLevel(nivel)
```

```
logger.addHandler(memoria)
```

```
return memoria
```

```
def obtener_logger(nombre: str) -> logging.Logger:
```

```
    """Devuelve un logger hijo del arbol `featsel`."""
```

```
    return logging.getLogger(f"featsel.{nombre}")
```